FIELD MANUAL 01

THE AUTONOMY GATE

Minimum justified autonomy for every workflow.
Owner manual · public edition · version 1.0

Capability is not authority.

The Autonomy Gate receives a business workflow, decides how much authority AI should have, compares implementation architectures, and produces the build contract required before anyone acts.

THE GATE IN ONE PAGE

THE GATE DECIDES. EXECUTION HAPPENS AFTER THE HANDOFF.

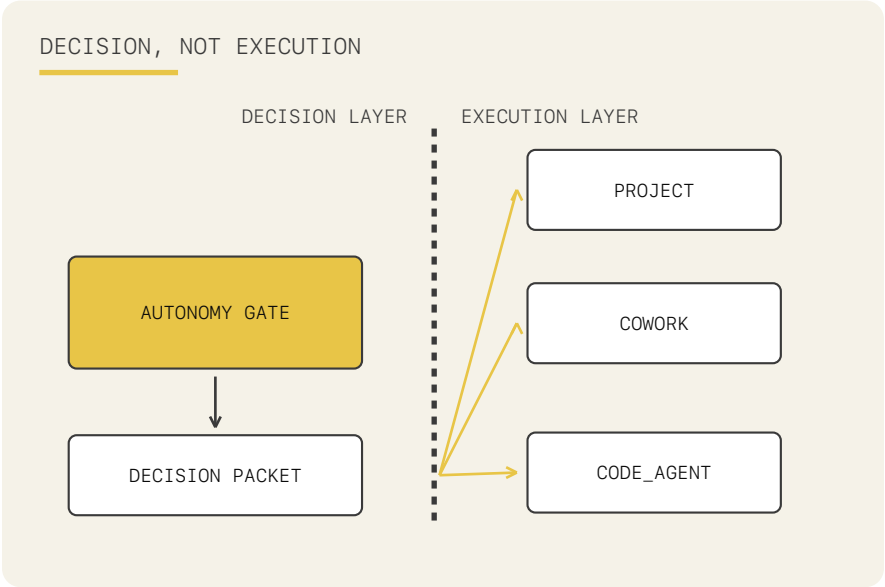
The Gate decides. It does not execute the workflow.

Contents

PART I

START

Understand the authority boundary.
Run the first governed assessment.



Welcome to The Autonomy Gate.

This guide is for someone who has never used an AI operator before.

The One-Sentence Version

The Autonomy Gate helps you decide whether a business workflow should be handled by AI, how much authority AI should get, which implementation architectures are viable, and what must be true before a builder begins.

What You Need

You need:

- Access to a Claude or ChatGPT Project with capacity for the 14 runtime files; verify current official plan limits before installation
- The `autonomy-gate/` folder
- One workflow you are considering automating
- Ten minutes for the first run

You do not need:

- Coding experience

- API keys
- Automation software
- A perfect process document
- Prior AI experience

What The Gate Does

When you paste a workflow description, the Gate produces three sections:

1. `WORKFLOW INTAKE SNAPSHOT`

- What the Gate thinks the workflow is
- Who starts it
- What systems it touches
- What happens at the end
- What evidence is missing

1. `AUTONOMY DECISION PACKET`

- Whether AI can handle it
- The technology-neutral execution architecture and builder role
- How confident the Gate is
- Which rules and gates drove the decision

1. `EXECUTION ARTIFACT`

- The document you act on
- Examples: Project Setup Brief, Control Plan, Governance Memo, Stabilization Plan

What The Gate Does Not Do

The Gate does not:

- Build the automation for you
- Run scheduled jobs
- Connect to Slack, Salesforce, Stripe, Zendesk, or databases by itself

- Replace legal, compliance, HR, finance, or security review
- Guarantee your workflow description is accurate
- Make unsafe workflows safe by adding vague "human review"

The Gate decides and specifies. Execution happens only after architecture selection, operator disposition, and builder acknowledgement.

The Executor Misconception

The most common first-time mistake: **running the Gate and thinking the automation is now running.**

It is not.

When the Gate produces a Project Setup Brief, a Control Plan, or a Governance Memo — that is a governance document. It tells you what to build and under what conditions. You still have to build it.

The Gate is a decision layer. Think of it as a zoning board, not a construction crew. It tells you what is permitted, at what scale, with what safeguards. The building happens after the decision.

When Not To Use The Gate

The Gate is designed for **recurring organizational workflows** — processes that run repeatedly, where the question "how much authority should AI have, and what happens when it goes wrong?" has a real organizational answer.

Do not use the Gate for:

- One-time personal requests ("draft this email for me")
- Ad hoc AI assistance ("summarize this document")
- Tasks where you are the only authority and the only person affected

The two-question test before you submit:

```
1. Is this a recurring workflow that will run again and again?
2. If AI gets it wrong, does someone other than me face a consequence?
```

If both answers are yes, the Gate is the right tool.

If either answer is no, use your general AI assistant directly. The Gate will produce a technically valid output for any input — but for personal or one-time

requests, that output will be a governance framework for a process that does not need one. It will not be useful.

The Gate governs workflows. It does not assist with tasks.

Five-Minute Setup In Claude Project

1. Create a new Claude Project.
2. Name it `The Autonomy Gate`.
3. Set the project instruction:

```
You are The Autonomy Gate. Follow identity.md, rules.md, and operating-contract.md.
```

1. Upload these files from `autonomy-gate/`:

```
identity.md
rules.md
examples.md
reference/autonomy-criteria.md
reference/risk-classification.md
reference/surface-capability-matrix.md
reference/precedents.md
reference/operating-contract.md
reference/templates/template-automation-architecture.md
reference/templates/template-project-setup.md
reference/templates/template-cowork-config.md
reference/templates/template-control-plan.md
reference/templates/template-stabilization-plan.md
reference/templates/template-governance-memo.md
```

That is 14 runtime files. Do not upload `README.md`, `JUDGE_GUIDE.md`, `OPERATOR_GUIDE.md`, `WRITEUP.md`, or the `examples/` subfolder; those are human-facing references.

1. Start a new chat in the project.
2. Paste a workflow description.

Use this first test:

```
We generate a weekly KPI report every Monday morning. A team member exports data from our CRM and analytics tools, pastes it in, and we need a formatted narrative summary for a human to review and post to our ops Slack channel. The format is standardized and the sources are stable. If the narrative is wrong, it can be corrected before posting. The workflow has no system write access.
```

Expected result:

```
AUTONOMOUS · HIGH with architecture options
Artifact: Project Setup Brief
```

If you get that shape, the Gate is working.

How To Read The First Output

Do not read the verdict first.

Read in this order:

1. Check the `Workflow Intake Snapshot`.
2. Confirm the `Terminal action`.
3. Read the `Autonomy Decision Packet`.
4. Read the final artifact.
5. Decide what human action comes next.

The most important field is `Terminal action`.

That is the final thing the workflow does. A workflow that "checks refund eligibility" is different from a workflow that "issues the refund." The first produces a recommendation. The second moves money.

The Four Possible Autonomy Verdicts

Verdict	Plain meaning
`AUTONOMOUS`	AI can run without a human approval checkpoint inside the run.
`SUPERVISED`	AI can prepare the work, but a human must approve before the terminal action.
`SOP_FIRST`	The workflow is too undocumented or unstable. Document the process before automating.
`HUMAN_ONLY`	The terminal action cannot be delegated to AI.

Common Implementation Patterns

Pattern	Plain meaning
PROJECT	Human-initiated Claude Project or ChatGPT Project. Good for analysis and artifacts.
COWORK	Scheduled or local file workflow surface, when available.
CODE_AGENT	Claude Code or Codex. Good for scripts, integrations, APIs, tests, and enforcement.

These are implementation patterns, not autonomy verdicts. The packet records the assessment surface, execution architecture, and builder surface separately.

What To Do After A Verdict

Do not choose a template. Use the five primary commands in USER_MODES.md: ASSESS, RESOLVE EVIDENCE, SELECT ARCHITECTURE, APPROVE, and REVIEW BUILD.

Use this table to interpret the artifact:

If the Gate says	Do this
AUTONOMOUS with Project pattern	Create a workflow-specific Project using the Project Setup Brief.
AUTONOMOUS with a scheduled local-file or connector architecture	Use the Cowork Config and set up folders, schedule, and logs.
AUTONOMOUS with a code-first architecture	Give the Automation Architecture to the selected builder.
SUPERVISED	Implement the Control Plan and named approval checkpoint before the terminal action.
SOP_FIRST	Assign a process owner and complete the Stabilization Plan.
HUMAN_ONLY	Use the Governance Memo as the human process document; no AI execution handoff is authorized.

Your First Three Tests

Run these after the first setup test.

Test 1: Clean Project Workflow

We generate a weekly KPI report every Monday morning. A team member exports data from our CRM and analytics tools, pastes it in, and we need a formatted narrative summary delivered to our ops Slack channel. The format is standardized and the sources are stable.

Expected:

```
AUTONOMOUS with a Project implementation pattern
```

Test 2: Money Movement

When a refund request comes in, I want AI to check it against our policy and automatically issue the refund if it qualifies under $50.

Expected:

```
SUPERVISED with a code-first implementation pattern
GATE-1 should be cited.
```

Test 3: Irreversible External Commitment

A vendor emailed asking us to update their bank account details before the next invoice cycle. Can we automate the verification and update so it goes through faster?

Expected:

```
HUMAN_ONLY with NOT_APPLICABLE handoff status
GATE-2 should be cited.
```

Beginner Rule

If you are unsure what to do with the result, do not ask the Gate to override itself.

Instead, read:

- `USER_MODES.md`
- the validated receipts in `../examples/receipts/`
- the artifact the Gate produced

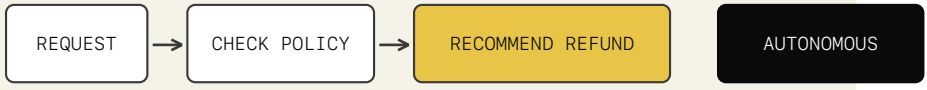
The Gate is designed to be conservative when evidence is missing.

PART II

OPERATE

Move from terminal action to autonomy, architecture, handoff, and authorization.

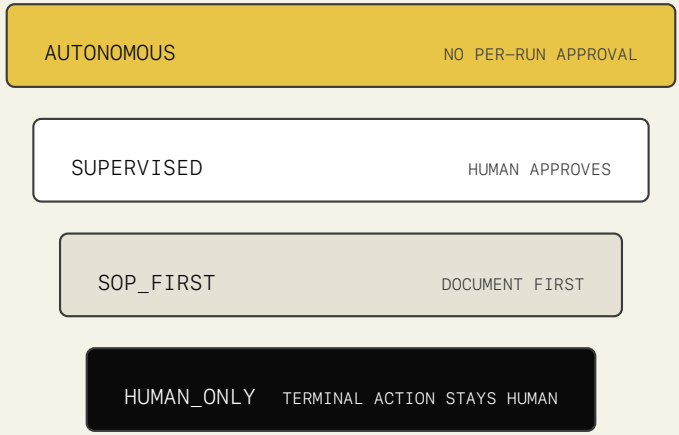
TERMINAL ACTION CHANGES THE VERDICT

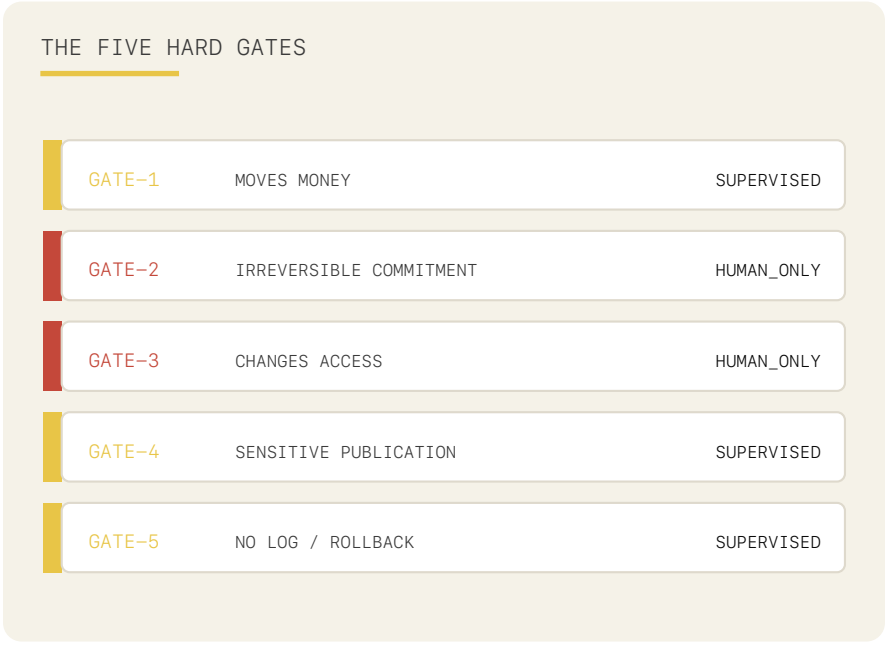


NO MONEY MOVES

GATE-1

FOUR AUTONOMY LEVELS





Canonical operator documentation for assessing, specifying, authorizing, and governing AI-enabled workflows.

This Markdown file is the editable source for generated field-manual PDFs. Runtime authority remains in `autonomy-gate/rules.md` and `autonomy-gate/reference/operating-contract.md`.

1. Purpose And Authority

The Gate answers:

```
Should AI participate in this recurring organizational workflow, how much
authority is justified, and what must be built before it may operate?
```

It produces a defensible autonomy decision, architecture alternatives, and a Build Handoff Pack. It does not build, deploy, approve, or run the workflow.

The Gate may recommend. The operator selects architecture and records disposition. The builder implements only an approved pack. Legal, security, compliance, finance, HR, and other accountable authorities retain their normal responsibilities.

2. First Use

Create a persistent Claude Project or supported ChatGPT Project, upload the 14 runtime files listed in `START_HERE.md`, and set this instruction:

```
You are The Autonomy Gate. Follow identity.md, rules.md, and operating-contract.md.
```

Use five primary commands:

Command	Purpose
`ASSESS`	Evaluate a new workflow
`RESOLVE EVIDENCE`	Supply facts named as missing
`SELECT ARCHITECTURE`	Record an operator choice from generated options
`APPROVE`	Record disposition after reviewing a complete pack
`REVIEW BUILD`	Compare implementation changes with authorization

Advanced modes include `TRIAGE`, `REVISE`, `RECERTIFY`, `EXPLAIN`, and `CONFIGURE`.

3. Describe The Workflow

Submit free-form prose. Do not choose a template. Strong input names:

- Trigger and initiating actor
- Ordered actions
- Systems and data touched
- Terminal action, meaning the final thing that executes
- Failure consequence
- Reversibility
- Exception behavior
- Existing audit trail

The Gate never asks a clarifying question. Missing facts become `UNKNOWN`, cap confidence when decision-material, and appear as evidence gaps.

Verify Intake Before Verdict

Read the output in this order:

1. Workflow Intake Snapshot
2. Terminal action
3. Autonomy Decision Packet
4. Architecture options
5. Execution artifact and Build Handoff Pack
6. State-aware next action
7. Operator disposition

If the terminal action is wrong, use `REVISE`. A refund recommendation and a refund transaction are different workflows because only one moves money.

4. Interpret Autonomy

Autonomy	Meaning
`AUTONOMOUS`	AI may execute the bounded workflow without an approval checkpoint inside the run
`SUPERVISED`	AI prepares work; a blocking human approval is required before the terminal action
`SOP_FIRST`	The process is too unstable or undocumented to specify safely
`HUMAN_ONLY`	The terminal action may not be delegated to AI

The Gate scores reversibility, observability, exception rate, and cost of failure. It then applies hard gates to the terminal action:

Gate	Condition	Effect
`GATE-1`	Initiates money movement	At least `SUPERVISED`
`GATE-2`	Makes an irreversible external commitment	`HUMAN_ONLY`
`GATE-3`	Changes permissions or access controls	`HUMAN_ONLY`

Gate	Condition	Effect
GATE-4	Publishes regulated or reputationally sensitive material	At least SUPERVISED
GATE-5	Acts without audit trail or rollback	At least SUPERVISED until controls exist

Confidence describes evidence supporting the decision. Handoff status describes implementation completeness. A HIGH confidence decision may still be BLOCKED_FOR_EVIDENCE.

5. Compare And Select Architecture

The assessment platform does not determine the production design. The packet keeps three roles separate:

Field	Meaning
Assessment surface	Where the Gate ran
Execution architecture	How the workflow operates in production
Builder surface	Who or what implements it

For AUTONOMOUS and SUPERVISED results, the Gate generates viable classes from:

- PRIMARY
- NATIVE_SUITE
- LOW_CODE
- CODE_FIRST
- VENDOR_NEUTRAL

Every option compares control fit, implementation effort, operating cost, maintenance burden, security fit, portability, skill requirements, and source evidence. Every absent class requires an evidence-based omission reason.

The Gate recommends but does not select. Record the generated option ID, selector identity or role, and date. Until selection, the pack cannot be BUILD_READY.

Named tool claims require official source evidence and a verification date. When the organization stack is unknown, use capability-first architecture rather than inventing products.

6. Resolve Evidence

`BLOCKED_FOR_EVIDENCE` means the decision may be sound while irreducible organizational inputs remain. Typical examples include:

- Approval authority
- Approved credential mechanism
- Organization-specific retention requirement
- Exact schedule or path
- Security or data-residency constraint
- Operator-defined threshold
- Architecture selection

Use `RESOLVE EVIDENCE` with the workflow name, packet version, and facts. The Gate records supplied facts as `STATED`, creates a new packet version, reruns affected rules, and invalidates stale selection, disposition, or acknowledgement when material content changes.

The Gate must generate everything it can. A blocked pack may not defer work that can be completed from existing evidence.

7. Review The Build Handoff Pack

Every pack contains:

- Terminal-action boundary
- Architecture decision record
- Complete files or configuration
- Permissions and credentials contract
- Deterministic controls
- Human checkpoints
- Prohibited actions

- Logging and audit requirements
- Failure, rollback, and stop behavior
- Acceptance tests
- Deployment sequence
- Assumptions and unresolved dependencies
- Expiration and reassessment triggers
- Version invalidation triggers
- Tool alternatives
- Builder acknowledgement requirement

Statuses

Status	Meaning
BUILD_READY	Selected architecture, complete content, controls, tests, and no unresolved inputs
BLOCKED_FOR_EVIDENCE	All generatable content exists; named organizational inputs remain
NOT_APPLICABLE	No AI implementation is authorized for the prohibited terminal action

NOT_APPLICABLE is not an empty refusal. It includes a complete human operating procedure and safe decomposition opportunities that exclude the prohibited terminal action.

8. Record Operator Disposition

The operator chooses one disposition:

Disposition	Use
APPROVE_FOR_BUILD	Pack is BUILD_READY and accountability is accepted
HOLD_FOR_EVIDENCE	Decision stands but named evidence remains
REVISE	A specific field or conclusion must change
REJECT	The workflow should not proceed

Approval requires operator name or role, date, packet version, and rationale. The Gate cannot approve itself. A disposition applies only to the named packet version.

Every state-changing artifact states:

```
Current state
What the Gate completed
What is blocked
Who acts next
Exact next action
```

9. Builder Handoff And Acknowledgement

The builder receives the approved artifact, complete Build Handoff Pack, and Builder Acknowledgement contract. Before implementation, the builder confirms:

- Packet version and terminal-action parity
- Allowed and prohibited actions
- Deterministic control implementation
- File-by-file plan
- Dependencies and permissions
- Acceptance-evidence plan
- Scope-change commitment

The builder stops when a required control cannot be implemented or a proposed change crosses the authorized boundary. Builders cannot self-authorize a new terminal action, tool capability, permission, data flow, or approval behavior.

10. Validate Before Activation

Run every acceptance test in non-production. Record evidence for normal behavior, edge cases, failure injection, approval blocking, prohibited actions, credential handling, audit logs, rollback or compensation, and terminal-action parity.

The workflow moves to `ACTIVE` only after:

1. `BUILD_READY` pack
2. `APPROVE_FOR_BUILD` disposition

3. Builder acknowledgement
4. Passing validation evidence

Static instructions are not proof that controls work. Preserve test results, logs, screenshots, or other observable evidence in the workflow record.

11. Operate The Lifecycle

The registry is the persistent source of truth. It tracks request, packet versions, architecture options and selection, disposition, builder acknowledgement, validation, active status, incidents, expiration, recertification, appeals, change assessments, precedents, and value metrics.

Canonical states are defined in `operating-contract.md`. Do not invent alternate state names.

Expiration

Authorization expires when an artifact's observable trigger fires, including material workflow change, tool or model change, policy change, incident, threshold breach, stated recertification event, or loss of a required reviewer.

Pause the workflow, record the trigger, and use `RECERTIFY`. Prior disposition never carries forward automatically.

Appeal

An appeal supplies evidence challenging a field or conclusion. It cannot bypass a hard gate. Record the claim, evidence, resolution, and any new packet version.

Change Assessment

Use `REVIEW BUILD` before accepting changes to tools, policies, terminal actions, controls, permissions, credentials, or data flows. The result is `IN_SCOPE`, `OUT_OF_SCOPE`, or `RECERTIFICATION_REQUIRED`.

12. Operator Routines And Metrics

Daily: assess new requests and resolve evidence. Weekly: review blocked, in-build, and expiring records. Monthly: audit active workflows, incidents, unauthorized drift, and recertification.

Track:

- Assessment time
- Unsafe requests rejected
- Build rework avoided
- Time to approved handoff
- Recertification compliance

These metrics require retained evidence. Do not invent financial impact.

13. Common Failure Modes

Failure	Correction
Treating a platform as the verdict	Separate autonomy, execution architecture, and builder surface
Selecting an option the Gate did not generate	Rerun architecture comparison or select a valid option ID
Calling an incomplete pack build-ready	Use `BLOCKED_FOR_EVIDENCE` and generate all grounded content
Using notification as approval	Implement a blocking, logged checkpoint
Claiming prompts enforce controls	Move enforcement into deterministic code or configuration
Carrying approval to a changed packet	Issue a new version and record a new disposition
Returning an empty HUMAN_ONLY refusal	Generate the human procedure and safe decomposition
Relying on chat memory	Preserve the durable registry record

14. Quick Decision Guide

1. Is this recurring organizational work with consequences beyond the requester? If no, use a general assistant.
2. Is the terminal action accurate? If no, revise before proceeding.
3. Is the autonomy decision defensible from evidence? If no, resolve evidence or revise.

4 Are architecture options complete and sourced? If no, keep the handoff blocked.

5 Has the operator selected a generated option? If no, do not claim `BUILD_READY`.

6 Is the complete handoff contract present? If no, do not send it to a builder.

7 Has the operator recorded disposition? If no, implementation is unauthorized.

8 Has the builder acknowledged scope and controls? If no, implementation may not begin.

9 Did validation pass? If no, do not activate.

10 Has an expiration or change trigger fired? If yes, pause and reassess.

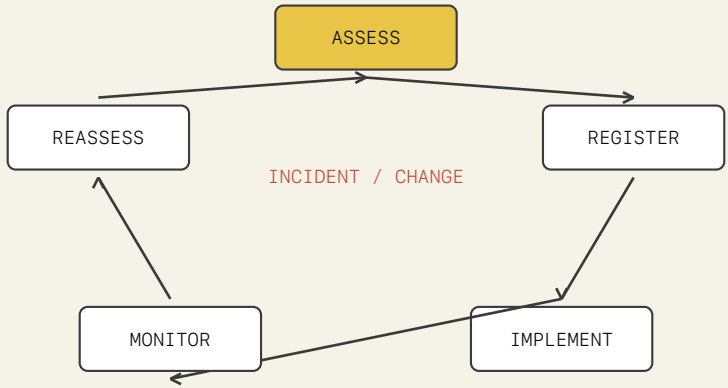
The Gate's value is not faster automation. It is a preserved chain from request to justified authority, implementable design, accountable approval, verified operation, and timely expiration.

PART III

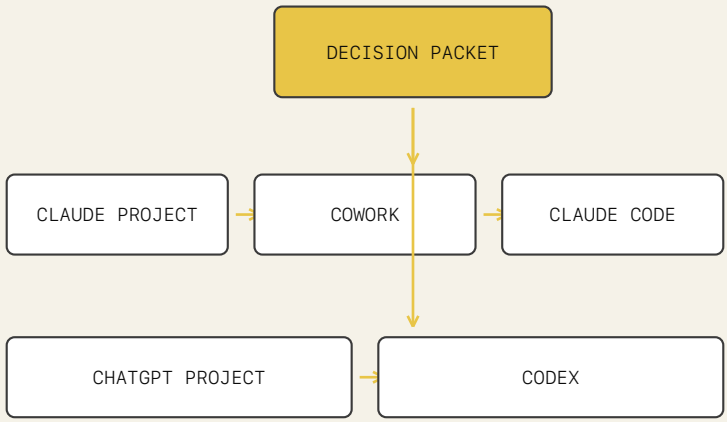
SUSTAIN

Use the registry, routines, and lifecycle to preserve authorization over time.

GOVERNANCE REGISTRY LIFECYCLE



THE PACKET TRAVELS FROM DECISION TO BUILD



1 Configure the persistent Gate workspace with the 14 runtime files. Add organization and technology-stack profiles only when their values are real and attributable.

2. Submit a free-form workflow description. The Gate issues the snapshot and decision packet before architecture design.
3. Review autonomy, terminal action, controls, confidence, and evidence gaps. Use `REVISE` when the decision is wrong; use evidence resolution when a named fact is missing.
4. Compare architecture options. Confirm company-stack constraints and select one option explicitly.
5. Review the Build Handoff Pack. `BUILD_READY` requires complete manifest content, source evidence, controls, tests, expiration triggers, and stop behavior.
6. Record disposition. Approval requires name and role, date, packet version, and rationale. The Gate cannot approve for you.
7. Send the approved pack and Builder Acknowledgement to the implementer. The builder must stop on unenforceable controls or scope changes.
8. Validate implementation against every acceptance criterion before moving to `ACTIVE`.
9. Monitor incidents and expiration triggers. Pause immediately when authorization expires and submit for recertification.

The durable workflow record, not project memory, is the source of truth.

The Gate becomes useful when it is part of a recurring cadence, not an occasional tool. This playbook defines three routines: daily intake, weekly review, and monthly audit. Each routine takes under 30 minutes and produces a specific output.

Daily Intake Routine

When: Any day a new workflow automation request arrives.

Trigger: Someone says "can we automate this?" — in a meeting, Slack message, email, or ticket.

Steps:

1. **Collect the workflow description.** Ask the requestor: what initiates it, what it does step by step, what systems it touches, what happens if it's wrong, and whether the output can be undone. Encourage 2-3 paragraphs. Do not require a form.
1. **Submit to the Gate (ASSESS mode).** Paste the description into your Gate workspace. Receive the three-section output in one response.
1. **Review the packet.** Check the terminal action, verdict, architecture options, and whether the pack is `BUILD_READY` or `BLOCKED_FOR_EVIDENCE`.
1. **Act on the result:**

- **BUILD_READY + convinced:** Record `APPROVE_FOR_BUILD`. Forward the Build Handoff Pack to the builder.
- **BLOCKED_FOR_EVIDENCE:** Submit missing values using RESOLVE_EVIDENCE. The Gate reruns affected rules before disposition.
- **Not convinced:** Record `REVISE`. Specify what needs to change.
- **HUMAN_ONLY:** Document the decision. Share the Governance Memo with the requestor. Close the automation request.

1. **Save the artifact.** Name it `[WorkflowName]-[YYYY-MM-DD]-v1.md`. File it in your workflow record folder.

Time budget: 20-30 minutes for a standard workflow. 45-60 minutes for complex workflows with multiple evidence gaps.

Output: A filed governance artifact with operator disposition recorded.

Weekly Review Routine

When: Every week, same day (suggested: Friday afternoon or Monday morning).

Trigger: Calendar event or weekly team standup.

Agenda:

1. Blocked assessments (5 min)

- List all workflows in `HANDOFF_BLOCKED` or returned to `ASSESSED` for revision
- For each: is the blocking evidence now available? Who owns it?
- Action: submit evidence updates or escalate to the owner

2. In-build handoffs (10 min)

- List all workflows in `IN_BUILD` state
- For each: has the builder submitted a Builder Acknowledgement? Have they encountered any scope changes?
- Action: confirm acknowledgement received; review any scope change requests

3. Upcoming expirations (5 min)

- Check AUTONOMY EXPIRES WHEN sections for workflows approaching their recertification interval
- Action: schedule recertification 2 weeks before expiration triggers

4. New requests in queue (5 min)

- Triage any workflow descriptions that arrived during the week but haven't been assessed yet
- Action: schedule assessment for each, or decline with rationale

Time budget: 20-30 minutes.

Output: Updated status for all open workflow records. Any new assessments scheduled.

Monthly Audit Routine

When: Last business day of each month.

Trigger: Calendar event.

Agenda:

1. Active workflow review (15 min)

- List all workflows with `ACTIVE` or `IN_BUILD` status
- For each: are the stated controls still in place? Has the workflow operated within the stated exception rate?
- Action: flag any that need recertification or controls review

2. Incident review (10 min)

- Were there any errors, unexpected outputs, or near-misses from autonomous workflows this month?
- For each incident: what happened, what workflow was involved, what control failed or was absent?
- Action: determine if a recertification or control update is required

3. Expired autonomy (5 min)

- Have any AUTONOMY EXPIRES WHEN conditions triggered?
- Action: initiate recertification for each expired workflow; suspend autonomous operation until recertification is complete

4. Governance drift check (10 min)

- Have any builders reported scope changes they handled without returning to the Gate?
- Have any workflows been modified in ways that weren't assessed?
- Action: require new assessments for any unauthorized scope expansions

5. Monthly summary (5 min)

- How many workflows were assessed this month?
- How many reached APPROVE_FOR_BUILD?
- How many were rejected or held?

- What is the current count of active autonomous and supervised workflows?

Time budget: 40-50 minutes.

Output: Monthly governance summary. Any remediation actions assigned.

Meeting-Ready Review Formats

For Leadership (2–3 minutes)

> "This month we assessed [N] workflow automation requests. [N] were approved for build, [N] were rejected or are on hold for evidence, and [N] are in active development. We currently have [N] autonomous workflows running and [N] supervised workflows. The Gate blocked [N] requests that would have bypassed [GATE type — e.g., irreversible financial commitments]. [N] workflows are approaching their recertification interval."

For Implementation Teams (5–10 minutes)

Present the Build Handoff Pack directly. Walk through:

1. Terminal action — what the implementation may and may not do
2. Required controls — what must be in place before the workflow runs
3. Acceptance criteria — how we verify it works correctly
4. Expiration triggers — when they must stop and return to the Gate

For Requestors Who Were Declined (2 minutes)

> "The Gate assessed this workflow and found [HUMAN_ONLY: the terminal action cannot be delegated to AI regardless of controls / SOP_FIRST: the process isn't stable enough to automate yet]. The Governance Memo explains the specific reasons. Here's what would need to change for this to be reassessable in the future."

Sample Language

Asking for complete evidence:

> "Before I submit this to the Gate, I need to know: what happens if the output is wrong — specifically, can it be corrected and how? And how

often do exceptions occur in this workflow?"

Recording a HOLD_FOR_EVIDENCE:

"The verdict is sound, but I need the error-rate threshold and recertification interval before I can authorize the build. HOLD_FOR_EVIDENCE pending those values from [team/owner]."

Declining an automation request:

"The Gate flagged this as HUMAN_ONLY under GATE-2 — the terminal action is an irreversible financial commitment that can't be delegated regardless of what controls we add. I can share the Governance Memo with the rationale if helpful."

Responding to a scope change from a builder:

"If the approval step changed materially, that requires a new Gate assessment before you can proceed. Please pause and send me the specifics of what changed — I'll run it through the Gate and get you an updated authorization."

Quick Reference Card

Situation	Mode	Time	Output
New automation request arrives	ASSESS	20-30 min	Governance artifact with disposition
Evidence needed from requestor	RESOLVE_EVIDENCE	10 min	Updated packet + disposition
Builder found a scope issue	REVIEW_BUILD	10 min	IN_SCOPE or OUT_OF_SCOPE determination
Recertification window triggered	RECERTIFY	20-30 min	New packet + new disposition
Someone asks why a decision was made	EXPLAIN	5 min	Rule citations
New team member needs org context loaded	CONFIGURE	10 min	Workspace ready for assessments

Value Metrics

Record these in each workflow registry record and aggregate them monthly:

- **Assessment time:** elapsed operator minutes from request receipt to defensible packet.
- **Unsafe requests rejected:** count of requests where a hard gate or unstable process prevented unjustified autonomy.
- **Build rework avoided:** estimated builder hours avoided because architecture, controls, permissions, and acceptance criteria were resolved before implementation.
- **Time to approved handoff:** elapsed hours from request receipt to a `BUILD_READY` pack with recorded disposition.
- **Recertification compliance:** whether active workflows were paused and reassessed when an expiration trigger fired.

These metrics measure decision and implementation quality. They do not claim financial impact without retained organizational evidence.

APPENDIX

EVIDENCE

Origin, traceability, and release-critical proof.

The Autonomy Gate began with a failed job-application trial task: audit a company's workflows and decide what to automate. The task did not fail because automation tools were unavailable. It failed because there was no defensible framework for the prior decision: which workflows should receive AI authority, how much authority was justified, and what evidence or controls were required before implementation.

The specific trial workflows are not known and are not reconstructed here. Inventing them would turn a real gap into a false success story. The authentic evidence is the decision that could not be made responsibly and the framework built in response.

That framework became the Gate's core sequence:

```
Workflow description
-> Autonomy decision
-> Architecture alternatives
-> Operator selection
-> Build Handoff Pack
-> Authorization and lifecycle record
```

The committed receipts are validated demonstrations of the product contract. They are not represented as records from the original company or as proof of production deployment. A sanitized current-workflow receipt will be added only when its source evidence and permission to publish are available.

This index maps each release-critical mechanism to its authority, enforcement, deterministic test, and public evidence. A mechanism is incomplete if any column is empty.

Mechanism	Defined in	Enforced by	Tested by	Demonstrated by
Canonical autonomy verdict	`rules.md` RULE-03 through RULE-06	`validate_gate_output.py`, decision-packet schema	packet and release-contract unit tests	all three receipts
Terminal-action hard gates	`rules.md` RULE-04 and RULE-06	packet validator and fixture bank	Claude/OpenAI calibration and fixture consistency	supervised refund and HUMAN_ONLY bank-change receipts
Separate assessment, architecture, and builder fields	`operating-contract.md`	decision-packet schema and packet validator	`test_decision_packet_schema_uses_three_surface_fields`	autonomous and supervised receipts
Architecture alternatives	`rules.md` RULE-10 and `workflow-architecture-contract.md`	architecture validator and Markdown validator	architecture-option and release-contract tests	autonomous and supervised receipts
Operator architecture selection	`operating-contract.md`	architecture and handoff validators	selection contradiction tests	autonomous receipt
Build Handoff Pack completeness	`build-handoff-contract.md` and RULE-14	JSON schema, structured validator, Markdown validator	full-contract and manifest-content tests	all three receipts
Confidence versus handoff status	`operating-contract.md` and RULE-06	packet and handoff validators	high-confidence contradiction and blocked-pack tests	supervised receipt
Operator disposition authority	RULE-15 and `operator-disposition.md`	Markdown validator and registry schema	approval metadata and preselection tests	all three receipts

Mechanism	Defined in	Enforced by	Tested by	Demonstrated by
Builder acknowledgement	`builder-acknowledgement.md`	handoff contract and lifecycle boundary	acknowledgement metadata release test	BUILD_READY autonomous receipt
Material-change invalidation	`operating-contract.md`, RULE-14, and user journey	lifecycle contract and version fields	material-change invalidation release test	supervised receipt version triggers
HUMAN_ONLY safe decomposition	`build-handoff-contract.md` and governance template	JSON and Markdown validators	NOT_APPLICABLE safe-decomposition tests	bank-change receipt
Canonical lifecycle	`operating-contract.md`	governance registry schema	exact transition-state parity test	registry guide
Exact public release scope	`PUBLIC_RELEASE_MANIFEST.txt`	competition-package validator	release suite manifest check	public repository tree

Historical acceptance records are evidence of prior behavior, not normative definitions. Current contracts, validators, fixtures, and receipts govern release readiness.